# 0045. Jump Game II

y: Medium | Category: Greedy | LeetCode: https://leetcode.com/problems/jump-g

# Problem

- Return the minimum number of jumps to reach the last index (always reachable).

# Intuition

- Implicit BFS: all indices reachable with j jumps form a contiguous window. The next window ends at the farthest point reachable from the current window.

# Brute Force

- DP: dp[i] = min over j < i reaching i of dp[j] + 1: O(n^2).

# Optimal Approach

- Jumps = 0, end = 0 (current window end), far = 0.
- For i in 0..n-2: far = max(far, i + nums[i]); if i == end: jumps++, end = far.

# Worked Example

```
[2,3,1,1,4].

i=0 far=2 i==end(0) -> jumps=1 end=2
i=1 far=4
i=2 far=4 i==end(2) -> jumps=2 end=4
answer 2
```

# Java Solution

```java
public class JumpGameII {
    public int jump(int[] nums) {
        int jumps = 0, end = 0, far = 0;
        for (int i = 0; i < nums.length - 1; i++) {
            far = Math.max(far, i + nums[i]);
            if (i == end) { jumps++; end = far; }
        }
        return jumps;
    }
}
```

# Complexity

- Time: O(n). Space: O(1).

# Pitfalls

- Loop to n-2: jumping from the last index would count an extra jump.